

Machine Learning A (2026–2027)

Home Assignment 3

Ludovico Maria Spitaleri, DMH249

Contents

1	Optimal Classification (16 points)	2
1.1	Question 1	2
1.2	Question 2	2
1.3	Question 3	3
2	Logistic Regression (50 points)	3
2.1	Cross-entropy error measure (16 points)	3
2.1.1	Rewriting negative logarithmic likelihood as cross-entropy	3
2.1.2	Cross-entropy and logistic regression	3
2.2	Logistic regression loss gradient (24 points)	3
2.2.1	Gradient for -1 and 1	3
2.2.2	Gradient for 0 and 1	3
2.2.3	Influence of misclassified examples	3
2.3	Log-odds (10 points)	3
3	Sleep Well (34 points)	3
3.1	Data understanding and preprocessing (4 points)	3
3.2	Classification (30 points)	4
3.2.1	Logistic regression	4
3.2.2	Random forest	4
3.2.3	Nearest neighbor	4
4	Overfitting (optional, 0 points)	4

1 Optimal Classification (16 points)

1.1 Question 1

Let us map the original label space $Y \in \{0, 1\}$ to a new space $Y' \in \{-1, 1\}$ following the original probabilities table for X and Y .

We define our decision function based on the original probabilities:

$$\delta'(x) = P(Y' = 1 \mid X = x) - P(Y' = -1 \mid X = x)$$

The classification rule in the mapped space is:

$$h'(x) = \text{sgn}(\delta'(x))$$

Evaluating this for the given inputs:

$$\begin{cases} \delta'(0) = 0 - 1 = -1 \Rightarrow h'(0) = \text{sgn}(-1) = -1 & \text{if } x = 0 \\ \delta'(1) = 0.2 - 0.8 = -0.6 \Rightarrow h'(1) = \text{sgn}(-0.6) = -1 & \text{if } x = 1 \end{cases}$$
$$\Rightarrow h'(x) = -1$$

Mapping back to the original space Y , the optimal classification rule in the original space is $h(x) = 0$.

The expected risk (0-1 loss) is the probability of misclassification:

$$\begin{aligned} R &= P(h(X) \neq Y) \\ &= \sum_{x \in X} P(X = x) P(h(x) \neq Y \mid X = x) \\ &= \sum_{x \in X} P(X = x) P(Y = 0 \mid X = x) \\ &= 0.5 \cdot 0 + 0.5 \cdot 0.2 = 0.1 \end{aligned}$$

1.2 Question 2

In the case of $X = 0$, the probabilistic classifier still always predicts the correct label $Y = 0$, since $P(Y = 0 \mid X = 0) = 1$.

On the other hand, when considering $X = 1$ the expected risk is given by the probability of a mismatch between the classifier and the true label:

$$\begin{aligned} &P(h(x) \neq Y \mid X = 1) \\ &= P(h(x) = 0 \mid X = 1) P(Y = 1 \mid X = 1) + P(h(x) = 1 \mid X = 1) P(Y = 0 \mid X = 1) \\ &= 0.8 \cdot 0.2 + 0.2 \cdot 0.8 = 0.32 \end{aligned}$$

The total expected risk is therefore:

$$\begin{aligned} R &= P(h(X) \neq Y) \\ &= \sum_{x \in X} P(X = x) P(h(x) \neq Y \mid X = x) \\ &= 0.5 \cdot 0 + 0.5 \cdot 0.32 = 0.16 \end{aligned}$$

1.3 Question 3

No, it is not possible to find a better classifier than the best one previously found, that is the deterministic classifier defined in section 1.1. This is because by mapping the label space to $Y' = \{-1, 1\}$ the risk of this classifier corresponds to the Bayes risk, known to be the minimum over all possible measurable functions h .

$$h^{\text{Bayes}}(x) = \text{sgn}[\mathbb{E}_p(Y \mid X = x)]$$

2 Logistic Regression (50 points)

2.1 Cross-entropy error measure (16 points)

2.1.1 Rewriting negative logarithmic likelihood as cross-entropy

2.1.2 Cross-entropy and logistic regression

2.2 Logistic regression loss gradient (24 points)

2.2.1 Gradient for -1 and 1

2.2.2 Gradient for 0 and 1

2.2.3 Influence of misclassified examples

2.3 Log-odds (10 points)

3 Sleep Well (34 points)

3.1 Data understanding and preprocessing (4 points)

The library pandas is used to load the datasets from the csv files. The classes frequencies are then calculated directly using numpy.

```
counts_train = np.bincount(y_train)
freq_train = counts_train / counts_train.sum()
```

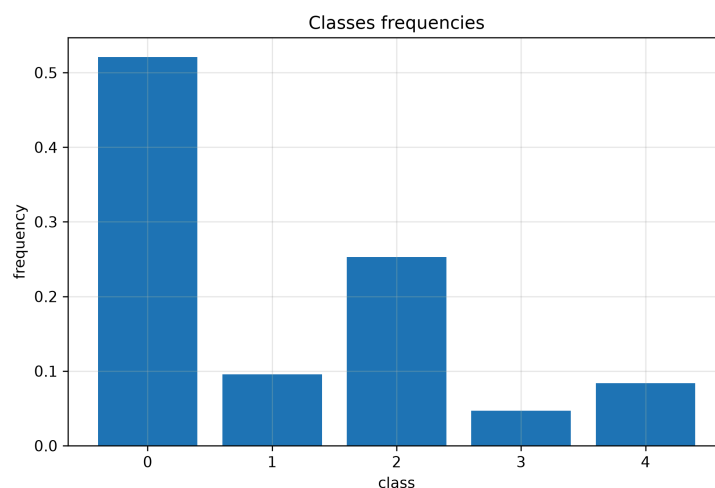


Figure 1: Classes frequencies bar chart

3.2 Classification (30 points)

3.2.1 Logistic regression

The LogisticRegression class from sklearn is used to fit a model to the training dataset. For both the train and test datasets, the prediction of the model is compared to the original labels to calculate the average error.

Regularization was not performed.

```
lr = LogisticRegression(max_iter=150).fit(X_train, y_train)
lr_train_err = np.mean(lr.predict(X_train) != y_train)
lr_test_err = np.mean(lr.predict(X_test) != y_test)
```

Errors:

- Training set: 0.1496
- Test set: 0.0993

3.2.2 Random forest

The RandomForestClassifier class from sklearn.ensemble is used to fit a model to the training dataset. For both the train and test datasets, the prediction of the model is compared to the original labels to calculate the average error. The out-of-bag error is instead calculated as $1 - \text{oob score}$.

Regularization was not performed.

```
rf = RandomForestClassifier(n_estimators=ntrees, oob_score=True).fit(X_train,
y_train)
rf_train_err = np.mean(rf.predict(X_train) != y_train)
rf_test_err = np.mean(rf.predict(X_test) != y_test)
rf_oob_err = 1 - rf.oob_score_
```

Errors:

Trees	Training err	Test err	OOB err
50	0.0004	0.1132	0.1519
100	$2.9652 \cdot 10^{-5}$	0.1101	0.1495
200	0	0.1099	0.1481

3.2.3 Nearest neighbor

4 Overfitting (optional, 0 points)